

VolSeg Documentation; Functionality

The following guides detail the full utilities and functions available with the Volume-Sgmantics toolkit. This section should be used in conjunction with the [Settings-Guide](#), where modes can be switched using the variables listed in the Training and Prediction .YAML files. This document is split into # sections for easier navigation including;

- Default Usage
- Training Command Split
- Epoch and Learning Rate Adaption
- Advanced funtions
- augmentation
- 2.5D training
- Multitask training; boundary map
- Self-supervised training; unlabel data: mean-teacher, pseudo-label

As newer versions of the toolkit are released, further settings may be added and as such this documentationguide will be updated.

Default Usage

When you have successfully installed Vol-Seg into its own environment, you will have access to its full functionality and can begin your training and prediction commands. It is important to view and curate your data before using the toolkit, as better data inputs equals better model outputs, and note that once you have completed a training model or prediction, you should rename and save your training models and predictions to a separate space away from the volume-segmantics directory; this is to stop you from overwriting your files with newly created ones and keep track of your segmentation history and used settings.

- Activate your VolSeg environment and Navigate to the VolSeg directory using the '*conda activate*' and '*cd*' commands.
- The '*model-train-2d*' and '*model-predict-2d*' commands can only be used once you have navigated to your Vol-Seg directory.

Please refer to the [ReadMe documentation](#) regarding basic command use and outputs, and read the *Default Parameter Walkthrough* in the [Setting Guide](#) before using the command line.

```
conda activate "path_to_env"
cd /users/'Individual_User'/"libs"/volume-segmantics

model-train-2d --data 'directory_location_image' --labels
'directory_location_labels'

model-predict-2d 'directory_location_segmanitics_training__model'
'directory_location_new_image'
```

Training can also be performed on multiple ROIs. several image-label pairs can be used in the training command though the image and the labels must be separated to correspond to their relevant dataloader. the images and labels must be input into the execution code in chronological order to keep the pairs together when training; failing to do this will result in a mismatch in image and label feature recognition and your resultant model will fail. An example format for multiple ROI training can be found below;

A model trained on multiple ROI pairs can be used to predict onto any larger image as per the normal utility without any additional inputs.

```
model-train-2d --data 'image_1_path.tiff' 'image_2_path.tiff' --labels
'label_1_path.tiff' 'label_2_path.tiff'
```

- Image_1 and label_1 are the first image-label pair, and image_2 and label_2 are the second. Each image and label must specify the full file path.

Split-Command Execution Shortcuts

When executing the *training* command, 2 processes occur: '**slicing**' the input data, and then '**training**' a model using that sliced data. If you plan to either execute multiple training runs on the same data slices with different training parameters, or execute training on predetermined slices, you can split the process and 'slice' the data once (saving it to the Vol-Seg directory in the process), and later run multiple 'train' commands on that same sliced data (reducing the overall segmentation process time). You can do this by defining the process you wish to occur using the following arguments; these are to be placed after the `--labels` argument in the training command

```
--mode=slicer

--mode=trainer
```

- e.g. `model-train-2d --data path_to_image_data --labels path_to_segmentation_labels --mode=slicer`

Running the '*slicer*' argument will create 2 folders within your working volume-segmentation directory: *data* and *seg*. This will contain the sliced data from the image and label inputs, of which the '*trainer*' will then use to execute the model training. The output sliced data and seg files can be copied and renamed for the purposes of reuse or streamlining a workflow; however, when the "--trainer" argument is executed, it will only train on the data within directory names 'data' and 'seg' within the Vol-Seg installation.

Be aware that if there are no such directories with these names within your Vol-Seg directory, the training argument will produce an error when executed. The data and seg folders produced by the '*slicer*' argument are **overwritten** if a full training (slicing and training through the basic command) is executed.

Epoch and Learning rate; Adapting Training Parameters per case

Epochs

TBC - being summerised

Learning Rate

TBC - being summerised

Advanced Functions; Training and Prediction Utilities and Settings setup

The following section explains the more advanced capabilities and functionalities within the toolkit, gives guidance on creating and linking additional data for these utilities to improve segmentation success and gives instructions for VolSeg settings editing relative to user case.

Augmentation settings

When you train a model, the trainer looks for comparable differences between what *is* (label-layer) and *is not* selected (background) from your data; it specifically looks for what is different. Augmentation is used to modify the data to expose this difference (contrast, blur, rotation, mirroring etc.) more clearly without changing the data itself. The VolSeg package has the capability to use 2 Augmentation libraries;

Albumentations (widely used in industry and open-source projects) and **MONAI** (healthcare-imaging-specific framework for multi-dimensional image preprocessing).

- Further information regarding [Albumentations](#) and [MONAI](#) can be found at the following links.

Both packages work very well and have produced great outcomes on past and current projects. Users of VolSeg can select either library though may find that depending on their input images that one set of augmentations works better than the other. To switch between these libraries, use the `2d_model_train_settings.yaml` to assign your preference prior to execution using the `'augmentation_library'` argument; further information can be found in the [setting-guide documentation](#).

2.5D Training and Predicting

~~ 2.5D explanation~ include 3/5/7/9 choice explanation; TBC

Implementing 2.5D can have a greater effect on larger datasets, those with more inner-label image complexity (where the image labels highlight a high variability in image detail (more differences for the trainer to measure)), wider image contrast threshold, or with datasets that contain a lower number of sub-layers (1-3 label layers). Datasets with instability in overall image contrast or a larger number of label sub-layers can often produce good segmentations with this method though it may also produce instances of hashing artefacts, inconsistencies in boundary identification across sub-layers and under/over segmented regions where the in-label area is possibly too complex. It is apparent that using ROI predictions from the 2.5D method, alongside those produced from non-2.5D Vol-Seg training, towards larger-image models can improve overall segmentation quality when following the recommended iterative segmentation workflow.

To enable this feature, change the `'use_2_5d_slicing'` input to **True** in the `2d_model_train_settings.yaml`, and then assign the `num_slices` to your preferred setting; the higher the number, the more data is chosen when the middle slice is selected. When choosing this number, also make sure you use the correct `'slice_file_format'` as per the directions in the [setting-guide documentation](#).

It is very important to make sure you use the same settings in the `2d_model_predict_settings.yaml` as the `2d_model_train_settings.yaml`; training models using `use_2_5d_slicing: True` requires the

same argument when predicting using that model outcome. *A model that incorporates data from 2.5D prediction, but does not use 2.5D when training the next model does not require a true argument when predicting.*

Multi-task Training

~ **Still being tested; instructions on functionality in process of being written up.**

~~ Multi-task explanation~ MONAI only; TBC

The multi-task utility allows additional data to be considered and processed during model *training* using multiple tasks on label data (decoder); segmentation using this method uses **3+** components/processing tasks; your standard *image* and *label layers*, alongside additional training data. Currently, this functionality includes a label '*boundary map*', though there is also the capability of adding additional tasks in future updates or through developmental work. This boundary map is created from your original label data, following the edge of your label concisely; this label should be as complete as possible with respect to the original image as incomplete labels can lead to poor segmentation outcomes using this method. *The boundary map will output as a .tif file; to train a model using this map, the image and label files must also be .tif files.* To generate a boundary map, use the instructions below;

- Boundary map Creation;

To create a boundary map of your data, use the '[CalculateBoundaryMap.py](#)' script. This script can be found in the `##` folder in the docs directory [here](#); it can be copied and saved to your VolSeg directory or to an easily accessible folder in your user space.

Open your Vol-Seg environment, navigate to your Vol-Seg Directory (or where the [CalculateBoundaryMap](#) script is saved to) and use the following command to formulate and execute the script and create your file; this map then can be opened and viewed in Napari.

1 - Activate your VolSeg environment and Navigate to the Vol-Seg directory using the '*conda activate*' and '*cd*' commands; `conda activate "path_to_env/env_name"` and `cd /users/'Individual_User'/libs/volume-segmentics` (or alternative location)

- Make sure the '[CalculateBoundaryMap.py](#)' script is present in the folder you have navigated to, and that you have opened the file (VisualStudioCode or equivalent file viewer) to check its contents. Using this script may also require the additional installation of additional packages not installed when installing VolSeg; if this occurs, the package requirements will appear as error messages and *pip* can be used to install them to the VolSeg environment; the requirements can be found in the first part of the script.

2 - The full command should have the format; `python CalculateBoundaryMap.py "Path_to_Image_File" --thickness 'Integer' --min_component_size 'Integer'`. Use the command arguments '`--thickness`' and '`--min_component_size`' to designate the *Boundary thickness* and the *solitary Component Size (within the boundary perimeter) you wish to remove from your map surrounding the label boundary(smaller than the integer allocated)*; they should be used after the execution of the *python command* running the script;

- "`Path_to_Image_File`" should be the full path to the label files location. If command arguments are not used, defaults for '`--thickness`' and '`--min_component_size`' will be

used ('3' and '0' respectively)); this is a good place to start if you are unsure about the initial integer inputs and what the boundary map produces.

```
conda activate "path_to_env/Vol_Seg-env"
cd /users/'Individual_User'/libs/volume-segmentics

python CalculateBoundaryMap.py "Path_to_Image_File" --thickness 'Integer' --
min_component_size 'Integer'
```

It is recommended that before you use your boundary map, you view it in Napari overlapping your original image and label file used to create the boundary map. If the outcome is unsatisfactory; alter the argument integers for the script (increase or decrease the default boundary thickness or component size to confine/redefine the boundary further).

- Multitask training;

To run the Multitask training, use the same format for running a simple training model, but add an extra argument designating the boundary map as an extra task; in this case the task has the designation '2' ('0' is the image, and '1' is the labels)

```
model-train-2d --data 'Path_to_Image_File' --labels 'Path_to_Label_File' --task2
'Path_to_BoundaryMap_File'
```

MONAI augmentations should be selected when using Multitask training, making use of its libraries specifically designed to incorporate boundary maps alongside label data. Adding the boundary maps to your data (adding the extra task) will also not only increase the 'slicing' time required when running the model, but also may require a longer epoch number to allow for better training conditions depending on the number of label sub-layers connected to your image (more complex boundary map used during the multitask training).

It is also possible to run a Multitask training model on multiple ROIs; you must the same number of boundary maps as image-labels pairs within the training execution command. List the image, label and boundary map files in the same order per argument, mirroring the same format as explained in the default usage training instructions.

```
model-train-2d --data 'image_1_path.tiff' 'image-2_path.tiff' --labels
'label_1_path.tiff' 'label_2_path.tiff' --task2 'boundarymap_1_path.tif'
'boundarymap_1_path.tif'
```

- image_1, label_1 and BM_1 are one image-label-BM set, and image_2, label_2 and BM_2 are a second image-label-BM set.

Implementing Multitask training can have a greater affect on medium-sized datasets, those with complex boundaries or with datasets that contain a higher number of sub-layers; large datasets or those that require more ROIs may need larger computing capabilities. Depending on the number of sub-layers (large quantity)

and the quality of both the boundary map, respective to the original segmentation, the outcome predictions may sometimes vary; changing the `loss_weights` in the 'Multi-task Learning settings' can positively affect these outcomes depending on the issues that arise where favouring the boundary or segmentation data to suit can be experimented with.

It is apparent that using this method on larger image iterations (rather than initial ROIS), where the ground truth is more evolved or complete, or where the segmentation boundaries take precedence in your data quality, can prove more effective and increase the final prediction quality when following the recommended iterative segmentation workflow.

To enable this feature, change the `use_multitask` query to **True**, and then make sure the `num_tasks` reflects the number of tasks you will be using during the training setting; '2' will be image and label as default with the addition of an extra task ('2') for the boundary map.

- `decoder_sharing`; TBC
- `seg_loss_weight`; TBC
- `boundary_loss_weight`; TBC
- `task3_loss_weight`; TBC
- `boundary_loss_type`; TBC

Self-Supervised Training

~ **Still being tested; instructions on functionality in process of being written up.**

~~ Self-Supervised Training explanation~ 2 integrations?? TBC

The Self-Supervision utility allows additional data to be considered during model *learning* using further unrelated image data (encoder); Segmentation using this method uses additional unlabelled image data to provide further support when looking for differences during label-image data comparison. *The unlabelled data should not be the same as that included in the main image inputs when training a model using this function; the unlabelled data should be from the same or a comparable scan but not from the same ROI.* This data must also be generated prior to the training models execution; to generate unlabelled data, use the instructions bellow:

- Generate Unlabelled Data; (use for both Mean Teacher and Pseudo-labelling)

To create your unlabelled data, you only require an image file; this should be either an ROI generated from the original image, that contains no data from the image you wish to use as 'labelled data' within the model, or an ROI from another image that contains the same image components in the same state as the labelled data (same contrast ratio and component realisation). The unlabelled data should also be smaller than the labelled image data by a factor of 40-70%; this is only a recommendation and can be outside of this range if required.

To generate the unlabelled data, take your ROI image file and run Vol-Seg at default settings (no advanced settings or additions) on '*licer mode* ', only allocating the `--data` argument (no labels). This will slice the ROI data into a '*data directory*' within your Vol-Seg directory that can then be saved to your user space in an easily accessible folder.

```
conda activate "path_to_env/Vol_Seg-env"
cd /users/'Individual_User'/libs/volume-segmentics
```

```
model-train-2d --data 'Path_to_ImageROI_File' --mode=slicer
```

Once generated, it may be used for either/both *Mean Teacher* and *Pseudo-label* self-supervised training modes.

- Self-Supervised training;

Mean Teacher and *Pseudo-label* training settings are used to create self-supervised training models, allocating your unlabelled data to the argument `--unlabeled_data_dir` within the training execution and producing potentially better predictions depending on your input image and label quality and overall segmentation aims. The following command can be used to assign your unlabelled data to both functions of the self-supervised training;

```
model-train-2d --data 'Path_to_Image_File' --labels 'Path_to_Label_File' --  
unlabeled_data_dir='Path_to_Unlabeled_Data_dir'
```

Specific settings for the *Mean Teacher* and *Pseudo-label* settings can be found in the subsequent sub-section.

- Mean Teacher; Settings and Outcome

~Mean teacher explanation~ ; TBC

- Pseudo-label; Settings and Outcome

~Pseudo-label~ ; TBC